UCL DEPARTMENT OF COMPUTER SCIENCE

COMP0264: Generative Artificial Intelligence

Master Past Exam Revision Guide & Exhaustive Solutions

Compiling Examinations and Core Concepts for Cohorts 2025–2026

Prepared by **Antigravity AI** for UCL Computer Science

July 2026

Master Course Syllabus & Core Roadmap

This master revision guide compiles all examination papers and core theoretical foundations for UCL COMP0264 (Generative Artificial Intelligence), featuring **each question immediately followed by its exhaustive, step-by-step solution**:

Exam / Topic	Core Concepts and Key Formulations
Paper 1: Mock Variant A	Encoder/Decoder VAEs, Posterior Collapse, Transformers, LLMs, and Diffusion Models.
Paper 2: Mock Variant B	Generator/Discriminator VAEs vs GANs, Mode Collapse, Scaling Laws, and Alignment.
Appendix	Standard Formulas: Chain Rule ($N = 2, 3, \ldots, n$), Bayes' Rule, ELBO, Gaussian KL, Softmax Scaling, DPO Loss, Score Matching SDEs.

PAPER 1: COMP0264 Mock Examination Variant A

Question 1: Variational Autoencoders & Posterior Collapse [25 Marks]

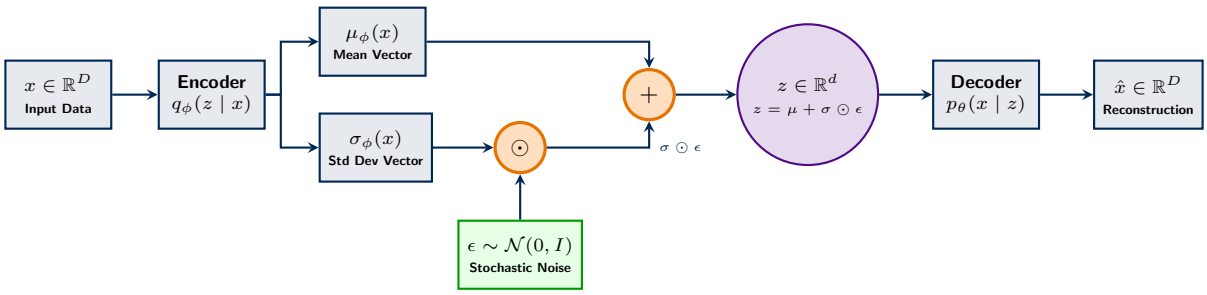
(a) Describe the architecture of the encoder and decoder of a Variational Autoencoder (VAE). Discuss the training procedure including the loss function (ELBO) used for optimization. [12.5 Marks]

(b) Explain why training VAEs can be unstable and discuss the causes of the potential problem of posterior collapse (i.e. the situation in which the encoder's output becomes almost identical to the prior, regardless of the input). Compare this to mode collapse in GANs and discuss possible solutions. [12.5 Marks]

Solution 1: Variational Autoencoders & Posterior Collapse

Core Concept Summary: VAEs optimize the Evidence Lower Bound (ELBO) using an encoder $q_\phi(z \mid x)$ and decoder $p_\theta(x \mid z)$. The Reparameterization Trick ($z = \mu + \sigma \odot \epsilon$) enables end-to-end gradient backpropagation through stochastic latent bottlenecks.

Perspective 1: Vector Computational Graph & Reparameterization Trick (TikZ Block Diagram)



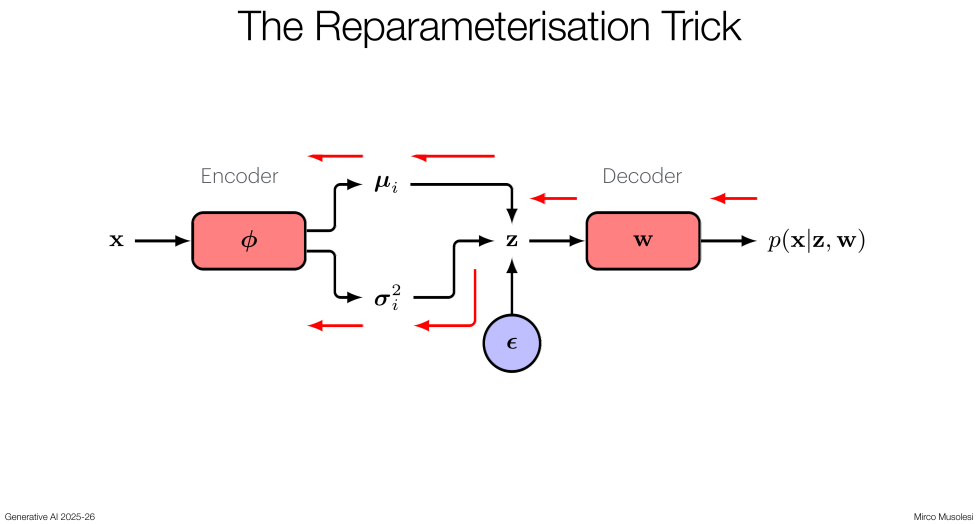


Figure 1: *Official Lecture Diagram: VAE Architecture & Reparameterization Trick.*

Perspective 2: Official Lecture Slide Architecture (COMP0264 Lecture 4, Slide 36)

(a) **Encoder/Decoder Architecture & ELBO Derivation (The WHY & Mechanics):**

- **Encoder** $q_\phi(z \mid x)$: Compresses high-dimensional data $x \in \mathbb{R}^D$ into mean $\mu_\phi(x)$ and variance $\sigma_\phi(x)^2$ parameterizing variational posterior $q_\phi(z \mid x)$.
- **Decoder** $p_\theta(x \mid z)$: Reconstructs original data $\hat{x}$ given latent sample $z \in \mathbb{R}^d$.

- **Why ELBO?**: Direct marginal log-likelihood $\log p_\theta(x) = \log \int p_\theta(x,z)dz$ is computationally intractable because integrating over all latent dimensions z cannot be done in closed form. ELBO provides a maximizeable surrogate lower bound.
- **Why Reparameterization Trick?**: Sampling $z \sim q_\phi(z \mid x)$ directly creates stochastic nodes that block backpropagation because expectations $\mathbb{E}_{z\sim q_\phi}[\cdot]$ cannot be differentiated directly with respect to encoder parameters ϕ. Isolating stochasticity into an exogenous noise variable $\epsilon \sim \mathcal{N}(0,I)$ via the deterministic transformation $z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$ makes z explicitly differentiable with respect to $\mu_\phi(x)$, $\sigma_\phi(x)$, and ϕ:

$$\frac{\partial z}{\partial \mu_\phi(x)} = 1, \quad \frac{\partial z}{\partial \sigma_\phi(x)} = \epsilon \implies \nabla_\phi z = \nabla_\phi \mu_\phi(x) + \epsilon \odot \nabla_\phi \sigma_\phi(x)$$

By the chain rule, gradients flow directly from the reconstruction loss through latent sample z into encoder parameters ϕ: $\nabla_\phi \mathbb{E}_{q_\phi}[\log p_\theta(x \mid z)] = \mathbb{E}_{\epsilon\sim\mathcal{N}(0,I)}\left[\nabla_z \log p_\theta(x \mid z) \cdot (\nabla_\phi \mu_\phi(x) + \epsilon \odot \nabla_\phi \sigma_\phi(x))\right]$.

Exhaustive 5-Step Derivation of the Evidence Lower Bound (ELBO): Our objective is to maximize the marginal log-likelihood of observed data $\log p_\theta(x) = \log \int p_\theta(x,z)dz$, which is computationally intractable due to the integral over all high-dimensional latent states z.

Step 1: Introduce Variational Distribution $q_\phi(z \mid x)$ & Expectation Form: We multiply and divide the integrand inside the marginal integral $\int p_\theta(x,z)\,dz$ by the encoder's variational posterior distribution $q_\phi(z \mid x)$:

$$\log p_\theta(x) = \log \int p_\theta(x,z)\,dz = \log \int q_\phi(z \mid x) \left[\frac{p_\theta(x,z)}{q_\phi(z \mid x)}\right] dz$$

Why does this integral equal an Expectation ($\mathbb{E}_{q_\phi}$)? By definition, for any continuous random variable $Z \sim q(z)$ with probability density function $q(z)$ and any arbitrary payload function $g(z)$, the expected value is defined as:

$$\mathbb{E}_{q(z)}[g(z)] \equiv \int q(z)\,g(z)\,dz$$

By identifying the density as $q(z) = q_\phi(z \mid x)$ and the payload function as $g(z) = \frac{p_\theta(x,z)}{q_\phi(z|x)}$, the integral $\int q_\phi(z \mid x)\left[\frac{p_\theta(x,z)}{q_\phi(z|x)}\right] dz$ matches the expectation definition $\mathbb{E}_{q(z)}[g(z)]$ exactly:

$$\log p_\theta(x) = \log \mathbb{E}_{q_\phi(z|x)}\left[\frac{p_\theta(x,z)}{q_\phi(z \mid x)}\right]$$

Step 2: Apply Jensen's Inequality: Because the logarithm $\log(\cdot)$ is a strictly concave function, pushing the expectation outside the logarithm yields a strict lower bound ($\log \mathbb{E}[Y] \geq \mathbb{E}[\log Y]$):

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p_\theta(x,z)}{q_\phi(z \mid x)}\right] \equiv \mathcal{L}_{\text{ELBO}}(\theta,\phi)$$

Step 3: Algebraic Expansion into Reconstruction & Regularization Terms: Factorize the joint distribution $p_\theta(x,z) = p_\theta(x \mid z)p(z)$ and split the logarithm ($\log \frac{A\cdot B}{C} = \log A + \log \frac{B}{C}$):

$$\begin{aligned}
\mathcal{L}_{\text{ELBO}}(\theta,\phi) &= \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p_\theta(x \mid z)\,p(z)}{q_\phi(z \mid x)}\right] \\
&= \mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x \mid z)\right] + \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p(z)}{q_\phi(z \mid x)}\right] \\
&= \mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x \mid z)\right] - \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{q_\phi(z \mid x)}{p(z)}\right] \\
&= \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)]}_{\text{Reconstruction Loss (Decoder)}} - \underbrace{KL(q_\phi(z \mid x) \parallel p(z))}_{\text{KL Divergence Regularization (Encoder)}}
\end{aligned}$$

Step 4: Exact Identity & KL Divergence Tightness: By Bayes' rule $p_\theta(z \mid x) = \frac{p_\theta(x,z)}{p_\theta(x)}$, substituting $p_\theta(x,z) = p_\theta(x)\,p_\theta(z \mid x)$ into the expectation yields the exact identity:

$$\log p_\theta(x) = \mathcal{L}_{\text{ELBO}}(\theta,\phi) + KL(q_\phi(z \mid x) \parallel p_\theta(z \mid x))$$

Since $KL(q \parallel p) \geq 0$, maximizing $\mathcal{L}_{\text{ELBO}}(\theta,\phi)$ directly minimizes the KL divergence to the true posterior!

Crucial Notation Distinction ($p_\theta(z \mid x)$ vs $p_\theta(x \mid z)$):

- **Decoder Network** $p_\theta(x \mid z)$: The neural network block shown in the architecture diagram that maps latent sample $z \in \mathbb{R}^d$ to data reconstruction $\hat{x} \in \mathbb{R}^D$.
- **True Latent Posterior** $p_\theta(z \mid x)$: The theoretical, intractable distribution over latents z given data x, defined via Bayes' rule as $p_\theta(z \mid x) = \frac{p_\theta(x|z)p(z)}{p_\theta(x)}$.
- **Why it MUST be $p_\theta(z \mid x)$ in the KL gap**: The KL divergence $KL(q_\phi(z \mid x) \parallel p_\theta(z \mid x))$ measures how closely the encoder $q_\phi(z \mid x)$ approximates the true posterior $p_\theta(z \mid x)$. Taking $KL(q_\phi(z \mid x) \parallel p_\theta(x \mid z))$ is mathematically impossible due to domain space mismatch ($z \in \mathbb{R}^d$ vs $x \in \mathbb{R}^D$).

Step 5: Final Neural Network Training Loss: In neural network gradient optimization, we minimize the negative ELBO ($\mathcal{L}_{\text{VAE}} = -\mathcal{L}_{\text{ELBO}}$):

$$\mathcal{L}_{\text{VAE}}(\theta, \phi) = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] + KL(q_\phi(z \mid x) \parallel p(z))$$

(b) **Training Instability, Posterior Collapse vs Mode Collapse (The WHY & Solutions):**

- **Why Training is Unstable**: VAE optimization balances two opposing forces: reconstruction loss (forcing z to capture data details) vs KL regularization (forcing $q_\phi(z \mid x) \to \mathcal{N}(0, I)$). Improper weighting causes gradient competition.
- **Why Posterior Collapse Occurs**: When paired with an expressive autoregressive decoder (e.g. PixelCNN or Transformer), the decoder can predict x_t using only past tokens $x_{<t}$ without referencing z. The network takes a shortcut: it sets $q_\phi(z \mid x) = p(z)$ to collapse $KL \to 0$ and ignores latent code z completely.
- **Posterior Collapse (VAE) vs Mode Collapse (GAN)**:
  - *Posterior Collapse (VAE)*: Encoder ignores input x, producing uninformative prior outputs $p(z)$. The decoder generates diverse samples, but lacks latent controllability.
  - *Mode Collapse (GAN)*: Generator collapses to outputting samples from only a few modes of P_{data}, missing entire regions of the data distribution.
- **Solutions to Posterior Collapse**: β-annealing (gradually increasing KL weight β from $0 \to 1$), Free Bits / KL Thresholding ($\min(KL, \tau)$), or discrete vector quantization (VQ-VAE).

Question 2: Transformer Architectures & Scaled Self-Attention [25 Marks]

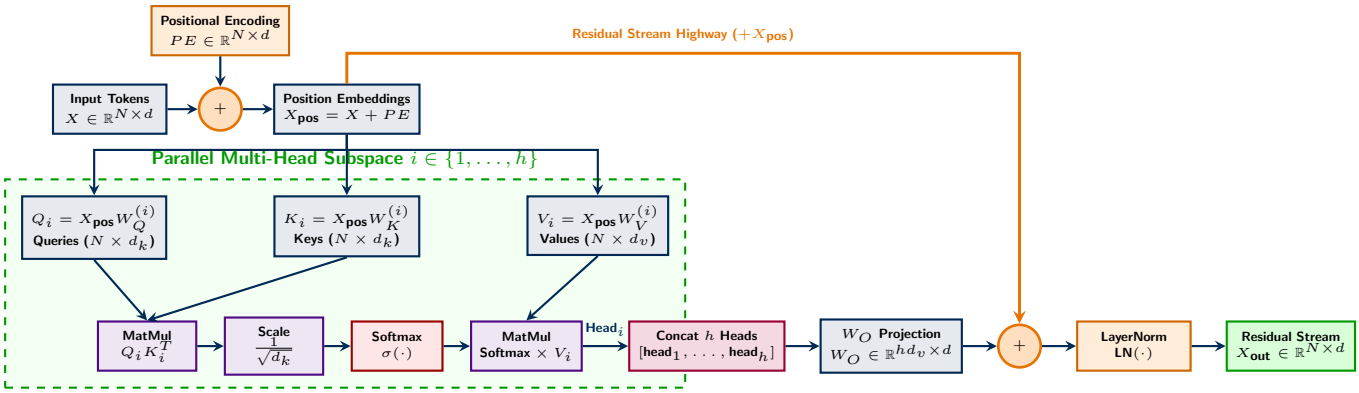
(a) Explain the self-attention mechanism in transformer architectures and why it is effective compared to recurrent neural networks. [5 Marks]

(b) Why is positional encoding necessary in transformers? Describe one method for incorporating positional information (e.g. sinusoidal or RoPE). [10 Marks]

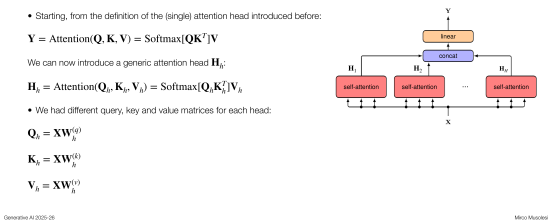
Solution 2: Transformer Architectures & Scaled Self-Attention

Core Concept Summary: One-Liner Definition: Self-Attention allows every token in a sequence to dynamically look at every other token, calculate how relevant they are ($\mathbf{QK}^T$), and pull in their most useful information ($\mathbf{V}$) to update its own meaning in context.

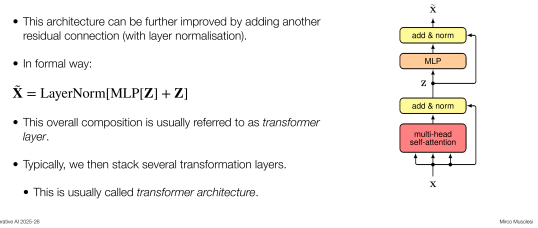
Perspective 1: Multi-Head Scaled Self-Attention Architecture (Top-to-Bottom Input Flow, Rightward Output Flow)



Multi-Head Attention



Transformer Layer and Transformer Architecture



(a) Multi-Head Self-Attention (Slide 31) **(b) Full Transformer Layer (Slide 42)**

Figure 2: *Official Lecture Diagrams: Multi-Head Self-Attention and Full Transformer Architecture.*

Perspective 2: Official Lecture Multi-Head Self-Attention & Layer Architecture (COMP0264 Lecture 2, Slides 31 & 42)

(a) **Self-Attention Mechanism vs RNNs (The WHY & Effectiveness):**

- **Why Self-Attention vs RNNs?**: RNNs process tokens sequentially ($O(N)$ sequential steps), causing a severe memory bottleneck for long sequences and preventing GPU parallelization. Self-Attention computes pairwise interactions between all tokens simultaneously in $O(1)$ path length, enabling full parallelization and direct routing.
- **Complete Matrix Breakdown: What are $\mathbf{X}, \mathbf{Q}, \mathbf{K}, \mathbf{V}$?**: All four matrices represent different transformations of the prompt tokens:
  (i) **Input Matrix $\mathbf{X} \in \mathbb{R}^{N\times d}$ (Raw State)**: The original token embeddings (static or output from previous layer) before head projection.

(ii) **Query Matrix $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q \in \mathbb{R}^{N \times d_k}$ (Search Question)**: Row $\mathbf{q}_i$ represents *what token i is actively looking for* in the sequence (e.g., verb "sat" asking "where is my subject noun?").

(iii) **Key Matrix $\mathbf{K} = \mathbf{X}\mathbf{W}_K \in \mathbb{R}^{N \times d_k}$ (Matching Index)**: Row $\mathbf{k}_j$ represents *what attributes token j advertises for matching* (e.g., "cat" advertising "I am a singular subject noun").

(iv) **Value Matrix $\mathbf{V} = \mathbf{X}\mathbf{W}_V \in \mathbb{R}^{N \times d_v}$ (Content Payload)**: Row $\mathbf{v}_j$ represents *the actual semantic content* that token j transfers if matched.

Why decouple $\mathbf{W}_K$ and $\mathbf{W}_V$?: Features needed to **route attention** (e.g., grammatical tags like "singular subject") are completely different from features needed to **update token representations** (e.g., semantic meaning "feline animal"). Decoupling $\mathbf{W}_K$ and $\mathbf{W}_V$ allows separate, unconstrained linear subspaces for routing vs content.

- **Does $\mathbf{QK}^T$ Draw Attention to Emphasize Useful Words?**: Yes! The dot product $\mathbf{QK}^T$ computes pairwise similarity scores between all $N \times N$ token pairs. If token i ("sat") has high alignment with token j ("cat"), $\mathbf{q}_i \cdot \mathbf{k}_j$ produces a large positive score. Applying $\mathbf{A} = \text{softmax}\left(\frac{\mathbf{QK}^T}{\sqrt{d_k}}\right)$ turns scores into probabilities summing to 1.0, **amplifying the most useful words** (e.g. 80% weight on "cat") while suppressing irrelevant noise. Multiplying by $\mathbf{V}$ ($\mathbf{O} = \mathbf{AV}$) blends the most useful words' content directly into token i's updated representation!
- **The Residual Stream in Transformers**: Yes! The Transformer relies on a central **Residual Stream** (information highway) $X \in \mathbb{R}^{N \times d}$. Multi-Head Attention acts as a "read/write" operation on this stream:

$$X_{\text{mid}} = \text{LayerNorm}\Big(X + \text{MultiHeadAttention}(X)\Big)$$

$$X_{\text{out}} = \text{LayerNorm}\Big(X_{\text{mid}} + \text{FFN}(X_{\text{mid}})\Big)$$

Adding the attention output back into X preserves gradient flow across deep layers.

- **Role of Linear Projections (W_O vs W_{lm} LM Head)**:

(i) **Attention Output Projection Matrix $W_O \in \mathbb{R}^{hd_v \times d}$**: Projects the concatenated outputs of all h parallel heads $[\text{head}_1, \ldots, \text{head}_h]$ back into the model's hidden dimension d so it can be added to the residual stream. It does *not* compute token probabilities.

(ii) **Unembedding Matrix / LM Head $W_{\text{lm}} \in \mathbb{R}^{d \times |V|}$**: Located at the **very end of the LLM**. It projects the final residual state $h_T \in \mathbb{R}^d$ onto the full vocabulary $|V|$ (e.g., $128,000$ tokens):

$$\text{logits} = h_T W_{\text{lm}} \in \mathbb{R}^{|V|}, \quad P(x_{T+1} \mid x_{\leq T}) = \text{softmax}(\text{logits})$$

This is the projection that computes token probabilities to predict the next word!

Formal Transformer Dimension Table (Color-Coded Numerical Example): Consider a fully consistent architecture: **Sequence Length $\mathbf{N = 512}$ tokens, Hidden Embedding Size $\mathbf{d = 4096}$, Heads $h = 32$, Projection Size $\mathbf{d_k = 128}$**:

Concrete 3-Token Worked Numerical Calculation ($\mathbf{N = 3, d = 4, d_k = 2}$): Let an input prompt contain $N = 3$ tokens: $\mathbf{x}_1$ = "The", $\mathbf{x}_2$ = "cat", $\mathbf{x}_3$ = "sat" with embedding matrix $\mathbf{X} \in \mathbb{R}^{3 \times 4}$:

$$\mathbf{X} = \begin{pmatrix} 0.0 & 1.0 & 0.5 & 0.5 \\ 1.0 & 2.0 & 0.0 & 2.0 \\ 0.0 & 3.0 & 1.0 & 0.0 \end{pmatrix}, \quad \mathbf{W}_Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{pmatrix}, \quad \mathbf{W}_K = \begin{pmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{W}_V = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}$$

1. **Queries, Keys, and Values Projection**:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q = \begin{pmatrix} 0.0 & 1.0 \\ 1.0 & 2.0 \\ 0.0 & 3.0 \end{pmatrix} \begin{matrix} \mathbf{q}_1 \text{ ("The")} \\ \mathbf{q}_2 \text{ ("cat")} \\ \mathbf{q}_3 \text{ ("sat")} \end{matrix}, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K = \begin{pmatrix} 0.5 & 0.5 \\ 0.0 & 2.0 \\ 1.0 & 0.0 \end{pmatrix} \begin{matrix} \mathbf{k}_1 \text{ ("The")} \\ \mathbf{k}_2 \text{ ("cat")} \\ \mathbf{k}_3 \text{ ("sat")} \end{matrix}$$

$$\mathbf{V} = \mathbf{X}\mathbf{W}_V = \begin{pmatrix} 0.5 & 1.5 \\ 1.0 & 4.0 \\ 1.0 & 3.0 \end{pmatrix} \begin{matrix} \mathbf{v}_1 \text{ ("The")} \\ \mathbf{v}_2 \text{ ("cat")} \\ \mathbf{v}_3 \text{ ("sat")} \end{matrix}$$

2. **Raw Pairwise Dot-Products $\mathbf{QK}^T$:**

$$\mathbf{QK}^T = \begin{pmatrix} 0.0 & 1.0 \\ 1.0 & 2.0 \\ 0.0 & 3.0 \end{pmatrix} \begin{pmatrix} 0.5 & 0.0 & 1.0 \\ 0.5 & 2.0 & 0.0 \end{pmatrix} = \begin{pmatrix} 0.5 & 2.0 & 0.0 \\ 1.5 & 4.0 & 1.0 \\ 1.5 & 6.0 & 0.0 \end{pmatrix}$$

3. **Variance Scaling by $\frac{1}{\sqrt{d_k}} = \frac{1}{\sqrt{2}} \approx 0.707$:**

$$\mathbf{S} = \frac{\mathbf{QK}^T}{\sqrt{2}} = \begin{pmatrix} 0.354 & 1.414 & 0.000 \\ 1.061 & 2.828 & 0.707 \\ 1.061 & 4.243 & 0.000 \end{pmatrix}$$

4. **Softmax Attention Weights Matrix $\mathbf{A} = \mathbf{softmax}(\mathbf{S})$:**

$$\mathbf{A} = \begin{pmatrix} 0.22 & \mathbf{0.63} & 0.15 \\ 0.13 & \mathbf{0.77} & 0.10 \\ 0.04 & \mathbf{0.95} & 0.01 \end{pmatrix} \begin{array}{l} \text{Row 1: "The"} \rightarrow 63\% \text{ attention on "cat"} \\ \text{Row 2: "cat"} \rightarrow 77\% \text{ attention on itself} \\ \text{Row 3: "sat"} \rightarrow \mathbf{95}\% \text{ attention on subject "cat"} \end{array}$$

5. **Weighted Value Aggregation Output $\mathbf{O} = \mathbf{AV}$:**

$$\mathbf{O} = \begin{pmatrix} 0.22 & 0.63 & 0.15 \\ 0.13 & 0.77 & 0.10 \\ 0.04 & 0.95 & 0.01 \end{pmatrix} \begin{pmatrix} 0.5 & 1.5 \\ 1.0 & 4.0 \\ 1.0 & 3.0 \end{pmatrix} = \begin{pmatrix} 0.89 & 3.30 \\ 0.935 & 3.575 \\ 0.98 & 3.89 \end{pmatrix}$$

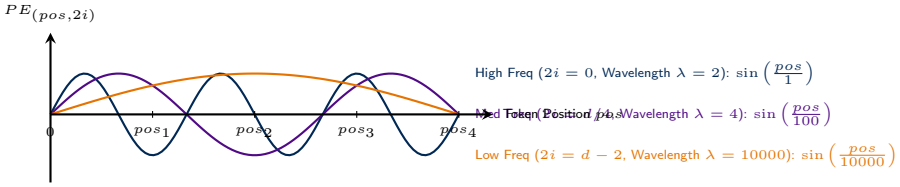
Notice how row 3 ($\mathbf{o}_{\text{sat}}$) successfully absorbs 95% of '"cat"'s semantic payload $\mathbf{v}_2 = (1.0, 4.0)$!

(b) **Positional Encodings (Sinusoidal & RoPE - The WHY):**

- **Why Positional Encoding is Necessary**: Matrix multiplication $\mathbf{QK}^T$ is permutation-equivariant — permuting input token order yields identical attention outputs. Positional encodings break this order-symmetry.

1. Absolute Sinusoidal Positional Encoding (Vaswani et al., 2017): For token position $pos \in \{0, 1, \ldots, N-1\}$ and dimension index $i \in \{0, \ldots, d/2 - 1\}$, the positional encoding vector $PE_{pos} \in \mathbb{R}^d$ is added directly to input embeddings ($X_{pos} \leftarrow X_{pos} + PE_{pos}$):

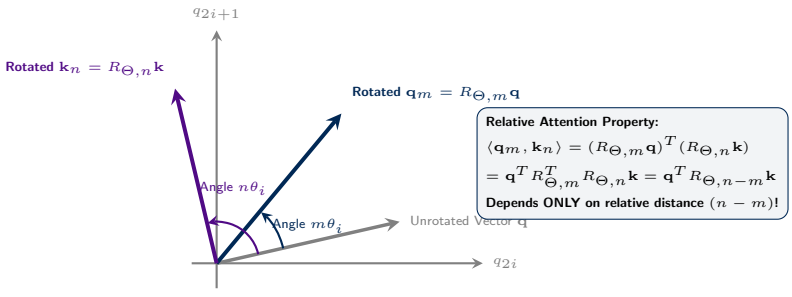
$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$



2. Rotary Position Embedding (RoPE - Su et al., 2021): Instead of adding position vectors to embeddings, RoPE multiplies Query/Key vectors by a 2D rotation matrix $R^{(i)}_{\Theta,m}$ for each 2D sub-vector pair (q_{2i}, q_{2i+1}) at token position m:

$$R^{(i)}_{\Theta,m} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix}, \quad \theta_i = 10000^{-2(i-1)/d}$$

$$\mathbf{q}^{(i)}_m = R^{(i)}_{\Theta,m}\mathbf{q}^{(i)}, \quad \mathbf{k}^{(i)}_n = R^{(i)}_{\Theta,n}\mathbf{k}^{(i)}$$



(c) **Variance Scaling Derivation ($\sqrt{d_k}$ - The WHY):**

Mathematical Step-by-Step Variance Scaling Derivation: Assume q_i and k_i are independent random variables with zero mean ($\mathbb{E}[q_i] = \mathbb{E}[k_i] = 0$) and unit variance ($\text{Var}(q_i) = \text{Var}(k_i) = 1$). Recall the fundamental relationship between variance and second moment:

$$\text{Var}(Z) = \mathbb{E}[Z^2] - (\mathbb{E}[Z])^2 \implies \mathbb{E}[Z^2] = \text{Var}(Z) + (\mathbb{E}[Z])^2$$

Therefore, the second moments (expected squared values) are:

$$\mathbb{E}[q_i^2] = \text{Var}(q_i) + (\mathbb{E}[q_i])^2 = 1 + 0^2 = 1, \quad \mathbb{E}[k_i^2] = \text{Var}(k_i) + (\mathbb{E}[k_i])^2 = 1 + 0^2 = 1$$

Since q_i and k_i are independent, $\mathbb{E}[q_i k_i] = \mathbb{E}[q_i]\mathbb{E}[k_i] = 0$, giving:

$$\text{Var}(q_i k_i) = \mathbb{E}[(q_i k_i)^2] - (\mathbb{E}[q_i k_i])^2 = \mathbb{E}[q_i^2]\mathbb{E}[k_i^2] - 0 = 1 \times 1 = 1$$

Summing over all d_k vector dimensions:

$$\text{Var}(\mathbf{q} \cdot \mathbf{k}) = \text{Var}\left(\sum_{i=1}^{d_k} q_i k_i\right) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = \sum_{i=1}^{d_k} 1 = d_k$$

Thus $\text{StdDev}(\mathbf{q} \cdot \mathbf{k}) = \sqrt{d_k}$. Dividing by $\sqrt{d_k}$ rescales variance $\text{Var}\left(\frac{\mathbf{q} \cdot \mathbf{k}}{\sqrt{d_k}}\right) = 1$, preventing extremely large dot products that would cause $\text{softmax}(\cdot)$ to saturate into one-hot vectors with near-zero gradients.

Question 3: Large Language Models (LLMs) [25 Marks]

(a) Despite being trained with a next-token prediction objective, LLMs exhibit emergent behaviors such as in-context learning (ICL). Provide a critical explanation, grounded in probabilistic modeling, of why such behavior arises. [5 Marks]

(b) In transformer-based LLMs, the conditional distribution $p(x_t \mid x_{<t})$ is parameterized using masked self-attention. Explain how masking ensures the autoregressive property is preserved during training. [5 Marks]

(c) Suppose an LLM assigns high probability to fluent but factually incorrect sequences. Explain why maximum likelihood training does not guarantee factual correctness, and discuss one potential solution (e.g. RLHF/DPO or RAG). [7.5 Marks]

(d) Explain how fine-tuning can lead to catastrophic forgetting from a probabilistic perspective. Use the Kullback-Leibler (KL) divergence to formalize the intuition. [7.5 Marks]

Solution 3: Large Language Models (LLMs) & Autoregressive Transformers

Core Concept Summary: In-Context Learning (ICL) acts as implicit Bayesian inference over latent tasks. Causal masking ($M_{ij} = -\infty$ for $j > i$) enforces autoregressive factorization. Preference alignment (DPO/RLHF) fixes MLE mode-covering hallucinations.

(a) **Emergent In-Context Learning (ICL) as Implicit Bayesian Inference (The WHY):** Despite being trained solely on next-token prediction without updating model weights ($\nabla_\theta = 0$), LLMs perform ICL. From a probabilistic perspective, pre-training data contains multi-task prompts. The prompt tokens $C = (x_1, y_1, \ldots, x_k, y_k, x_{\text{query}})$ provide evidence to infer a latent task concept variable τ:

$$P(y_{\text{query}} \mid x_{\text{query}}, C) = \sum_{\tau} P(y_{\text{query}} \mid x_{\text{query}}, \tau) P(\tau \mid C)$$

where posterior $P(\tau \mid C) = \frac{P(C|\tau)P(\tau)}{\sum_{\tau'} P(C|\tau')P(\tau')}$ updates beliefs over latent tasks via Bayes' rule.

Concrete Example of x, y, and Latent Task τ (Few-Shot Sentiment Analysis): Consider a 2-shot prompt context $C = (x_1, y_1, x_2, y_2, x_{\text{query}})$ fed to an LLM:

- **Demonstration 1**: x_1 = "I loved this movie!" $\rightarrow$ y_1 = "Positive" (Input text x_1, Output label y_1).
- **Demonstration 2**: x_2 = "Terrible plot, worst acting." $\rightarrow$ y_2 = "Negative" (Input text x_2, Output label y_2).
- **Query Input**: x_{query} = "A cinematic masterpiece." $\rightarrow$ $y_{\text{query}} = ?$

Bayesian Inference Process: Before seeing C, the pre-trained LLM has a prior distribution $P(\tau)$ over thousands of candidate tasks $\tau \in \{\text{Translation}, \text{Sentiment}, \text{Python Coding}, \text{Capital Cities}, \ldots\}$. As attention layers process (x_1, y_1) and (x_2, y_2), candidate tasks like *Translation* assign 0 likelihood to outputting "Positive/Negative". The posterior $P(\tau \mid C)$ collapses almost entirely onto τ = "Sentiment Classification". The model then evaluates $P(y_{\text{query}} \mid x_{\text{query}}, \tau = \text{Sentiment})$ to output y_{query} = "Positive"!

Terminology & Origin of y Values:

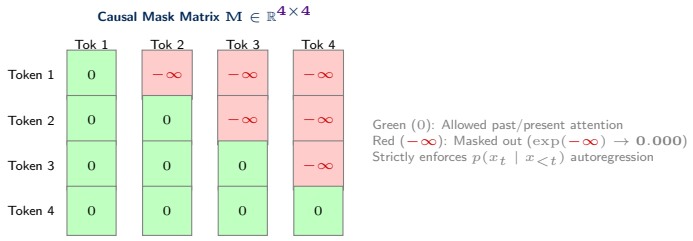
- **Formal Nouns for y**: Formally termed **Exemplar Outputs**, **Target Labels**, **Dependent Variables**, or **Ground-Truth Responses**.
- **Where do y Values Come From?**:
  (i) *Inference Time (Few-Shot Prompting)*: Provided directly by the **human user** in the prompt to demonstrate the desired transformation rule $x \rightarrow y$.
  (ii) *Pre-training Origin*: Learned from billions of naturally occurring structured (x, y) pairs on the web (e.g. StackOverflow Q&A $x \rightarrow y$, Wikipedia tables, GitHub code docstrings, bilingual translation corpora).

(b) **Causal Masking Matrix & Autoregressive Preservation (The WHY):** To parameterize $p(x_1, \ldots, x_T) = \prod_{t=1}^{T} p(x_t \mid x_{<t})$ in parallel during training, a causal mask matrix M_{ij} is added to

scaled dot-product attention scores before softmax:

$$\mathbf{S}_{\text{masked}} = \frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} + \mathbf{M}$$

Matrix Causal Mask Grid Diagram & Color-Coded Worked Numerical Example ($\mathbf{N = 4}$): For sequence length $\mathbf{N = 4}$ tokens, $M_{ij} = 0$ for $j \leq i$ (allowed attention) and $M_{ij} = -\infty$ for $j > i$ (masked future tokens):



Because $\exp(-\infty) = \mathbf{0}$, softmax converts all red entries to exactly **0.000** attention weight. Thus token t cannot attend to any future token $j > t$, preserving strictly valid probability factorization during parallel training.

(c) **Factual Correctness vs Maximum Likelihood Estimation (MLE) & DPO Solution (The WHY):** MLE training minimizes forward KL divergence between data and model distributions:

$$\mathcal{L}_{\text{MLE}} = KL(P_{\text{data}} \parallel P_\theta) = \mathbb{E}_{x\sim P_{\text{data}}}[-\log P_\theta(x)]$$

Because forward KL is **mode-covering**, model P_θ is heavily penalized for placing zero probability on valid text, forcing it to smooth probability mass across all plausible-sounding outputs, prioritizing fluency over factual truth (hallucinations).

Solution via Direct Preference Optimization (DPO): DPO optimizes model parameters θ directly on preference pairs (x, y_w, y_l) (preferred y_w vs dispreferred y_l) relative to a reference model π_{ref}:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

This penalizes ungrounded mode-covering hallucinations without requiring a complex RL reward model.

(d) **Probabilistic Formalization of Catastrophic Forgetting (The WHY):** When fine-tuning a pre-trained model θ_{pre} on a narrow dataset D_{ft}, likelihood optimization shifts the model distribution $P_{\theta_{\text{ft}}}$, causing the KL divergence $KL(P_{\theta_{\text{pre}}} \parallel P_{\theta_{\text{ft}}}) \to \infty$.
To prevent catastrophic forgetting of general reasoning capabilities, we add an explicit KL penalty to the fine-tuning loss:

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{FT}}(\theta) + \lambda \cdot \mathbb{E}_{x\sim D_{\text{pre}}}\left[KL(P_{\theta_{\text{pre}}}(\cdot \mid x) \parallel P_\theta(\cdot \mid x))\right]$$

This anchors fine-tuned policy P_θ close to pre-trained base distribution $P_{\theta_{\text{pre}}}$.

Question 4: Diffusion Models [25 Marks]

(a) Describe the forward and reverse processes in a diffusion model. What is the goal of each process? Formulate the underlying mathematical model for both processes and explain how the reverse process is derived from the forward process. [12.5 Marks]

(b) How is the reverse decoder in a diffusion model trained? What computational challenges arise in doing so, and what practical approach (ϵ-prediction MSE objective) is used instead? [12.5 Marks]

Solution 4: Diffusion Models & Score-Based Generative Frameworks

Core Concept Summary: Diffusion models define a forward noising chain $q(x_t \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I)$ and train a reverse network $\epsilon_\theta(x_t, t)$ to predict added noise via simplified MSE loss.

(a) **Forward Noising Chain and Reverse Denoising Process (Intuition, Verbal Explanation & Formal Models):**

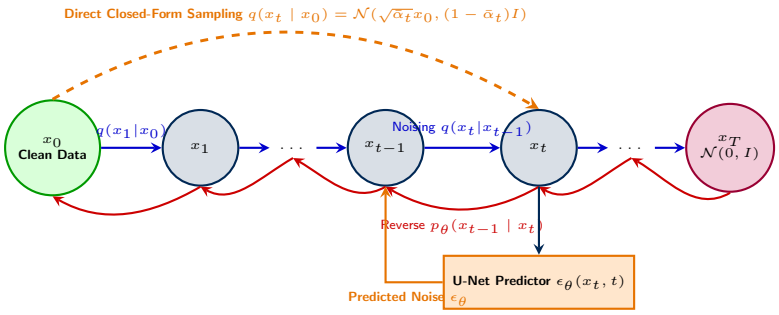
Layman Intuition & Real-World Analogies:

- **Forward Process (Adding Noise / Dissolving Ink in Water)**: Imagine dropping a single drop of blue ink (x_0) into a glass of clear water. Over time, ink particles diffuse randomly. At each small timestep, tiny random disturbances spread out the ink until the water becomes a uniform, cloudy blue blur ($x_T \sim \mathcal{N}(0, I)$). In image generation, the forward process systematically adds Gaussian noise step-by-step until the image is completely destroyed into featureless TV static.
- **Reverse Process (Removing Noise / Un-mixing Water)**: In real life, physics prevents cloudy water from spontaneously un-mixing back into a drop of ink. However, in generative AI, we train a neural network (e.g., U-Net) to observe a noisy image at step t and predict *which pixels were added as noise*. By subtracting a small fraction of that noise step-by-step, the network gradually reverses diffusion, turning pure random static back into a sharp, realistic photograph!

Verbal Explanation of Goals:

- **Goal of Forward Process**: Define a fixed, non-trainable Markov chain $q(x_t \mid x_{t-1})$ that systematically destroys structure in data $x_0 \sim p_{\text{data}}$ over T steps ($T = 1000$) using a noise schedule $\beta_1, \ldots, \beta_T$, driving x_T toward standard normal prior $\mathcal{N}(0, I)$.
- **Goal of Reverse Process**: Learn a parameterized Markov chain $p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$ that inverts each noising step, enabling sample generation by drawing $x_T \sim \mathcal{N}(0, I)$ and iteratively denoising back to x_0.

Standard Official DDPM Graphical Model & Architecture Diagram (Ho et al., 2020):



Detailed Mathematical Derivation of Direct Closed-Form Forward Sampling: 1. **Single Step Reparameterization**: Define variance schedule $\beta_t \in (0, 1)$ and set $\alpha_t = 1 - \beta_t$:

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(x_t; \sqrt{\alpha_t}x_{t-1}, (1-\alpha_t)I\right) \implies x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{1-\alpha_t}\epsilon_{t-1}, \quad \epsilon_{t-1} \sim \mathcal{N}(0, I)$$

2. **Recursive Substitution Back to x_0**: Substituting $x_{t-1} = \sqrt{\alpha_{t-1}}x_{t-2} + \sqrt{1-\alpha_{t-1}}\epsilon_{t-2}$ into x_t:

$$\begin{aligned} x_t &= \sqrt{\alpha_t}\left(\sqrt{\alpha_{t-1}}x_{t-2} + \sqrt{1-\alpha_{t-1}}\epsilon_{t-2}\right) + \sqrt{1-\alpha_t}\epsilon_{t-1} \\ &= \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \sqrt{\alpha_t(1-\alpha_{t-1})}\epsilon_{t-2} + \sqrt{1-\alpha_t}\epsilon_{t-1} \end{aligned}$$

Combining independent Gaussian distributions $\mathcal{N}(0, \alpha_t(1-\alpha_{t-1})I) + \mathcal{N}(0, (1-\alpha_t)I) = \mathcal{N}(0, (1-\alpha_t\alpha_{t-1})I)$:

$$x_t = \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \sqrt{1-\alpha_t\alpha_{t-1}}\bar{\epsilon}$$

3. **Closed-Form Direct Forward Sampling** $q(x_t \mid x_0)$: Defining cumulative product $\bar{\alpha}_t = \prod_{s=1}^{t}\alpha_s$:

$$q(x_t \mid x_0) = \mathcal{N}\left(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I\right) \implies x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Why this matters: Allows sampling noisy state x_t at any timestep t directly in $O(1)$ time without iterating through steps $1 \dots t-1$!

Detailed Mathematical Derivation of Reverse Process via Bayes' Rule: The true reverse distribution $q(x_{t-1} \mid x_t)$ is intractable because it depends on the unknown data distribution $p(x_0)$. However, conditioning on clean data x_0, Bayes' rule yields a tractable Gaussian posterior $q(x_{t-1} \mid x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I)$:

$$\begin{aligned} q(x_{t-1} \mid x_t, x_0) &= \frac{q(x_t \mid x_{t-1}, x_0)\, q(x_{t-1} \mid x_0)}{q(x_t \mid x_0)} \\ &\propto \exp\left(-\frac{1}{2}\left[\frac{(x_t - \sqrt{\alpha_t}x_{t-1})^2}{\beta_t} + \frac{(x_{t-1} - \sqrt{\bar{\alpha}_{t-1}}x_0)^2}{1-\bar{\alpha}_{t-1}}\right.\right. \\ &\qquad \left.\left. - \frac{(x_t - \sqrt{\bar{\alpha}_t}x_0)^2}{1-\bar{\alpha}_t}\right]\right) \end{aligned}$$

Expanding exponents and matching quadratic coefficients for x_{t-1} yields mean $\tilde{\mu}_t(x_t, x_0)$ and variance $\tilde{\beta}_t$:

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}x_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}x_t, \quad \tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$$

High-Scoring Exam Answer Template (Verbal Explanation with Minimal Equations):
Written Exam Strategy (How to get 100% marks using primarily text):

(a) **Define Forward Process**: State that the forward process is a fixed, parameter-free Gaussian Markov chain $q(x_t \mid x_{t-1})$ that gradually adds noise over T timesteps according to variance schedule β_t. Its goal is to transform complex real data x_0 into pure Gaussian noise $x_T \sim \mathcal{N}(0, I)$. Highlight the key property: using $\bar{\alpha}_t = \prod \alpha_s$, x_t can be sampled directly at any step t via $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$.

(b) **Define Reverse Process**: State that the reverse process inverts the noising trajectory to generate data. Since true $q(x_{t-1} \mid x_t)$ requires integrating over the unknown data distribution, we parameterize a Gaussian model $p_\theta(x_{t-1} \mid x_t)$ trained via a U-Net. Sampling begins by drawing $x_T \sim \mathcal{N}(0, I)$ and iteratively denoising down to x_0.

(c) **Derivation via Bayes' Rule**: Explain that conditioning on x_0 makes the reverse step tractable via Bayes' rule $q(x_{t-1} \mid x_t, x_0) = \frac{q(x_t|x_{t-1})q(x_{t-1}|x_0)}{q(x_t|x_0)}$. Because all three terms are Gaussians, completing the square proves that $q(x_{t-1} \mid x_t, x_0)$ is an exact Gaussian with mean $\tilde{\mu}_t$ depending on x_t and x_0.

(b) **Training the Reverse Decoder & ϵ-Prediction Objective (Computational Challenges & Detailed Derivations):**

Layman Intuition & Real-World Analogy (Predicting Mud vs Statue):

- **Direct x_0 Prediction (Hard)**: Imagine trying to guess what a marble statue looks like when it is buried under a huge mound of mud. Predicting the entire statue (x_0) directly in one step is extremely hard because there are infinitely many plausible statues.

- **Noise Prediction ϵ (Easy)**: Instead of predicting the whole statue, we train a neural network to predict *the exact shape of the mud added at that specific step* (ϵ). Once the network predicts the added noise, we subtract a small fraction of it to uncover a cleaner image!

Computational Challenges of Reverse Training:

(i) **High-Dimensional Intractability**: Direct maximum likelihood $\max_\theta \sum \log p_\theta(x_0)$ requires integrating out all intermediate noisy states $\int p_\theta(x_{0:T})dx_{1:T}$, which is computationally impossible.

(ii) **Non-Stationary Target Variance**: Predicting clean image x_0 directly from noisy state x_t is highly non-stationary across timesteps: early steps ($t \approx T$) have near-zero signal, while late steps ($t \approx 1$) have high signal, causing unstable neural network gradients.

Detailed Mathematical Derivation of ϵ-Prediction MSE Loss ($\mathcal{L}_{\text{simple}}$): 1. **Variational Lower Bound (ELBO)**: Minimizing negative log-likelihood $\mathcal{L}_{\text{VLB}}$ decomposes into KL divergence terms between Gaussian distributions:

$$\mathcal{L}_{\text{VLB}} = \mathbb{E}_q \Bigg[KL(q(x_T \mid x_0) \parallel p(x_T)) + \sum_{t=2}^{T} KL(q(x_{t-1} \mid x_t, x_0) \parallel p_\theta(x_{t-1} \mid x_t)) - \log p_\theta(x_0 \mid x_1)]$$

2. **Expressing Mean $\tilde{\mu}_t$ in Terms of Added Noise ϵ**: Using direct forward equation $x_t(\epsilon) = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon \implies x_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon}{\sqrt{\bar{\alpha}_t}}$, substitute x_0 into posterior mean $\tilde{\mu}_t(x_t, x_0)$:

$$\begin{aligned}\tilde{\mu}_t(x_t, x_0) &= \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}\left(\frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon}{\sqrt{\bar{\alpha}_t}}\right) + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}x_t \\ &= \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon\right)\end{aligned}$$

This motivates parameterizing our neural network mean $\mu_\theta(x_t, t)$ using noise-prediction network $\epsilon_\theta(x_t, t)$:

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right)$$

3. **Simplification to Unweighted MSE Objective**: Substituting $\tilde{\mu}_t$ and μ_θ into Gaussian KL divergence $KL(q \parallel p) = \frac{1}{2\sigma_t^2}\|\tilde{\mu}_t - \mu_\theta\|^2$ with noisy state $x_t(\epsilon) = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$:

$$\mathcal{L}_{t-1} = \mathbb{E}_{x_0,\epsilon}\left[\frac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar{\alpha}_t)}\left\|\epsilon - \epsilon_\theta\left(x_t(\epsilon),\, t\right)\right\|^2\right]$$

Ho et al. (2020) discovered that dropping weighting factor $\frac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar{\alpha}_t)}$ yields the **Simplified MSE Loss**:

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{t,x_0,\epsilon\sim\mathcal{N}(0,I)}\left[\left\|\epsilon - \epsilon_\theta\left(x_t(\epsilon),\, t\right)\right\|^2\right]$$

Why $\mathcal{L}_{\text{simple}}$ produces superior images: Dropping the weight factor down-weights loss at small t (where noise is small) and up-weights loss at large t, focusing network capacity on learning difficult global shape structures!

High-Scoring Exam Answer Template for Part (b) (Verbal Explanation with Minimal Equations): Written Exam Strategy for Part (b):

(a) **Explain Computational Obstacle**: State that training the decoder to predict x_0 directly is ill-conditioned and non-stationary. Early timesteps ($t \approx T$) contain almost zero image signal, while late timesteps ($t \approx 1$) are mostly clean image, causing extreme gradient scale variance.

(b) **Explain Noise Prediction Solution**: State that DDPM reparameterizes the posterior mean $\tilde{\mu}_t$ such that the neural network $\epsilon_\theta(x_t, t)$ only predicts the added noise vector $\epsilon \sim \mathcal{N}(0, I)$. Noise prediction turns training into a stationary regression problem with constant target variance (1.0).

(c) **Explain Loss Simplification**: State that while ELBO produces a weighted KL loss, Ho et al. (2020) showed that dropping the weighting coefficient yields $\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}[\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$. This simplified MSE objective down-weights small noise timesteps and up-weights large noise timesteps, prioritizing difficult global perceptually important features to achieve state-of-the-art sample quality.

PAPER 2: COMP0264 Mock Examination Variant B

Question 1: Generator/Discriminator GANs vs VAEs [25 Marks]

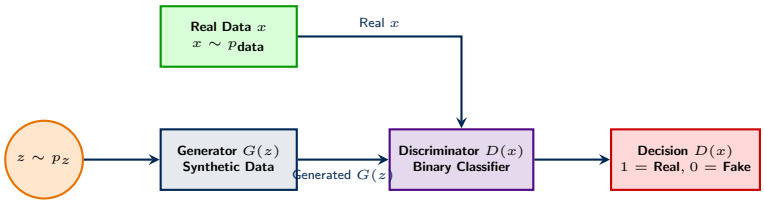
(a) Formulate the GAN minimax game $V(G,D) = \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1-D(G(z)))]$. Prove that for a fixed Generator G, the optimal Discriminator is $D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x)+p_g(x)}$. [12.5 Marks]

(b) Prove that when $D = D_G^*$, the minimax objective reduces to minimizing $2 \cdot JS(p_{\text{data}} \parallel p_g) - 2\log 2$. Discuss mode collapse and WGAN-GP. [12.5 Marks]

Solution 1: Generator/Discriminator GANs vs VAEs

Core Concept Summary: **Layman Analogy:** A GAN is an adversarial game between an Art Forger (Generator G) creating fakes from random noise z, and an Art Inspector (Discriminator D) learning to spot fakes. Optimal $D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x)+p_g(x)}$ collapses the game into minimizing Jensen-Shannon Divergence $JS(p_{\text{data}} \parallel p_g)$.

GAN Minimax Game Pipeline Architecture Diagram (TikZ Diagram):



(a) **Optimal Discriminator Derivation $D_G^*(x)$:** The minimax game objective $V(G,D)$ is defined as:

$$V(G,D) = \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1-D(G(z)))]$$

Writing the expectations explicitly in integral form:

$$V(G,D) = \int_x p_{\text{data}}(x)\log D(x)\,dx + \int_z p_z(z)\log(1-D(G(z)))\,dz$$

Applying the change of variables $x = G(z)$ with generated density $p_g(x)$, the second integral becomes $\int_x p_g(x)\log(1-D(x))\,dx$:

$$V(G,D) = \int_x \left[p_{\text{data}}(x)\log D(x) + p_g(x)\log(1-D(x))\right] dx$$

To maximize $V(G,D)$ with respect to $D(x)$ for a fixed G, we differentiate the objective pointwise inside the integral. Define function $f(y)$ for $y = D(x) \in (0,1)$:

$$f(y) = a\log y + b\log(1-y), \quad \text{where } a = p_{\text{data}}(x),\ b = p_g(x)$$

Taking the first derivative with respect to y and setting to zero:

$$\frac{df}{dy} = \frac{a}{y} - \frac{b}{1-y} = 0 \implies a(1-y) = by \implies a = (a+b)y \implies y^* = \frac{a}{a+b}$$

Taking the second derivative:

$$\frac{d^2f}{dy^2} = -\frac{a}{y^2} - \frac{b}{(1-y)^2} < 0 \quad (\text{since } a,b>0)$$

Because $f''(y) < 0$, $f(y)$ is strictly concave, confirming that y^* is a unique global maximum. Substituting $a = p_{\text{data}}(x)$ and $b = p_g(x)$ yields the optimal Discriminator:

$$D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}$$

(b) **Reduction to Jensen-Shannon Divergence, Mode Collapse, and WGAN-GP:**

Reduction to Jensen-Shannon Divergence Proof: Substitute optimal Discriminator $D_G^*(x)$ back into the objective function $V(G, D_G^*)$:

$$V(G, D_G^*) = \mathbb{E}_{x \sim p_{\text{data}}} \left[\log \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} \right] + \mathbb{E}_{x \sim p_g} \left[\log \frac{p_g(x)}{p_{\text{data}}(x) + p_g(x)} \right]$$

Multiply and divide the denominator inside both logarithms by 2:

$$\begin{aligned} V(G, D_G^*) &= \mathbb{E}_{p_{\text{data}}} \left[\log \left(\frac{1}{2} \cdot \frac{p_{\text{data}}(x)}{\frac{p_{\text{data}}(x) + p_g(x)}{2}} \right) \right] + \mathbb{E}_{p_g} \left[\log \left(\frac{1}{2} \cdot \frac{p_g(x)}{\frac{p_{\text{data}}(x) + p_g(x)}{2}} \right) \right] \\ &= \mathbb{E}_{p_{\text{data}}}[-\log 2] + KL \left(p_{\text{data}} \parallel \frac{p_{\text{data}} + p_g}{2} \right) + \mathbb{E}_{p_g}[-\log 2] + KL \left(p_g \parallel \frac{p_{\text{data}} + p_g}{2} \right) \\ &= -\log 2 - \log 2 + 2 \cdot JS(p_{\text{data}} \parallel p_g) = \mathbf{2 \cdot JS(p_{\text{data}} \parallel p_g) - 2 \log 2} \end{aligned}$$

Since $JS(p_{\text{data}} \parallel p_g) \geq 0$, the global minimum is achieved if and only if $p_g = p_{\text{data}}$, where $V(G^*, D^*) = -2 \log 2$.

Mode Collapse & Vanishing Gradients (The WHY): If data distributions p_{data} and p_g lie on disjoint low-dimensional manifolds in high-dimensional space ($\text{support}(p_{\text{data}}) \cap \text{support}(p_g) = \emptyset$), the optimal Discriminator reaches perfect accuracy $D^*(x) = 1$ for real data and $D^*(x) = 0$ for fake data. Then $JS(p_{\text{data}} \parallel p_g) = \log 2$ (constant), causing gradients to vanish ($\nabla_G V = 0$). The Generator collapses to producing only a few safe modes (**Mode Collapse**).

WGAN-GP Solution (Wasserstein GAN with Gradient Penalty): WGAN replaces JS divergence with Earth Mover's (Wasserstein-1) Distance $W(p_{\text{data}}, p_g) = \sup_{\|D\|_L \leq 1} \mathbb{E}_{p_{\text{data}}}[D(x)] - \mathbb{E}_{p_g}[D(x)]$ using Kantorovich-Rubinstein duality. To enforce the 1-Lipschitz constraint $\|D\|_L \leq 1$, WGAN-GP adds a gradient penalty term:

$$\mathcal{L}_{\text{WGAN-GP}} = \mathbb{E}_{p_g}[D(\tilde{x})] - \mathbb{E}_{p_{\text{data}}}[D(x)] + \lambda \mathbb{E}_{\hat{x} \sim p_{\hat{x}}} \left[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2 \right]$$

where $\hat{x} = \epsilon x + (1 - \epsilon)\tilde{x}$ interpolates between real and generated samples.

Question 2: LLM Scaling Laws & Preference Alignment [25 Marks]

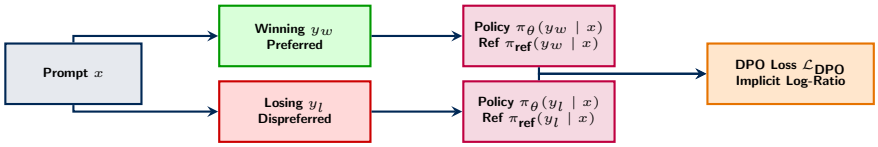
(a) Explain the Chinchilla scaling laws (Kaplan et al., Hoffmann et al.) governing compute $C \approx 6ND$, model parameters N, and dataset tokens D. [12.5 Marks]

(b) Derive the Direct Preference Optimization (DPO) loss from the Bradley-Terry preference model, explaining how DPO avoids training an explicit reward model. [12.5 Marks]

Solution 2: LLM Scaling Laws & Preference Alignment

Core Concept Summary: **Layman Analogy:** Chinchilla scaling proves that spending all your compute budget C building a massive model without enough training data is like buying a Ferrari without fuel ($N \propto \sqrt{C}, D \propto \sqrt{C}$). DPO reparameterizes implicit reward $r(x,y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$, enabling direct policy gradient optimization without a separate reward model.

DPO vs RLHF Implicit Reward Optimization Architecture (TikZ Diagram):



(a) **Chinchilla Scaling Laws Derivation (Kaplan vs Hoffmann):** Total pre-training compute C (in FLOPs) for a Transformer with N parameters trained on D tokens is approximated by $C \approx 6ND$.

Mathematical Derivation of Optimal Parameter-to-Token Ratio: Empirical cross-entropy loss $L(N, D)$ is modeled using power-law functions:

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$$

To find optimal $N^*(C)$ and $D^*(C)$ under a fixed compute budget $C = 6ND \implies D = \frac{C}{6N}$, substitute $D(N, C)$ into $L(N, D)$:

$$L(N) = E + AN^{-\alpha} + B\left(\frac{C}{6N}\right)^{-\beta} = E + AN^{-\alpha} + B \cdot 6^\beta C^{-\beta} N^\beta$$

Differentiating $L(N)$ with respect to N and setting derivative to zero:

$$\frac{\partial L}{\partial N} = -\alpha A N^{-\alpha-1} + \beta B \cdot 6^\beta C^{-\beta} N^{\beta-1} = 0$$

$$\alpha A N^{-\alpha-1} = \beta B \cdot 6^\beta C^{-\beta} N^{\beta-1} \implies N^{\alpha+\beta} = \left(\frac{\alpha A}{\beta B \cdot 6^\beta}\right) C^\beta$$

Solving for optimal parameters N^* and tokens D^*:

$$N^* \propto C^{\frac{\beta}{\alpha+\beta}}, \quad D^* \propto C^{\frac{\alpha}{\alpha+\beta}}$$

Hoffmann et al. (Chinchilla) empirically fitted $\alpha \approx 0.34, \beta \approx 0.28 \implies \frac{\alpha}{\alpha+\beta} \approx 0.5, \frac{\beta}{\alpha+\beta} \approx 0.5$. Thus, parameters N and tokens D should be scaled in equal proportions ($N \propto \sqrt{C}, D \propto \sqrt{C}$), requiring **$\sim$ 20 tokens per parameter** (e.g. 70B model trained on 1.4T tokens).

(b) **Complete Step-by-Step Derivation of DPO Loss from Bradley-Terry Model:**

1. Bradley-Terry Preference Model : Given prompt x and completions $y_w \succ y_l$, human preference probability is modeled via sigmoid of reward differences:

$$P(y_w \succ y_l \mid x) = \sigma(r(x, y_w) - r(x, y_l)) = \frac{1}{1 + \exp\left(-(r(x, y_w) - r(x, y_l))\right)}$$

2. Constrained RL Optimization Formulation : RLHF solves for optimal policy π_θ maximizing reward $r(x, y)$ subject to KL penalty against reference policy π_{ref}:

$$\max_{\pi} \mathbb{E}_{x\sim D, y\sim\pi} \left[r(x, y)\right] - \beta KL(\pi(y \mid x) \parallel \pi_{\text{ref}}(y \mid x))$$

$$\max_{\pi} \mathbb{E}_{x,y} \left[r(x, y) - \beta \log \frac{\pi(y \mid x)}{\pi_{\text{ref}}(y \mid x)}\right]$$

3. Closed-Form Optimal Policy Solution : Using Gibbs distribution properties, the exact closed-form optimal policy $\pi_r(y \mid x)$ is:

$$\pi_r(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y)\right), \quad Z(x) = \sum_y \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

4. Solving for Implicit Reward $r(x, y)$: Taking the logarithm on both sides of the optimal policy equation:

$$\log \pi_r(y \mid x) = \log \pi_{\text{ref}}(y \mid x) + \frac{1}{\beta} r(x, y) - \log Z(x)$$

$$r(x, y) = \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x)$$

5. Substituting Implicit Reward into Bradley-Terry Model : Substitute $r(x, y_w)$ and $r(x, y_l)$ into $P(y_w \succ y_l \mid x)$:

$$r(x, y_w) - r(x, y_l) = \left[\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} + \beta \log Z(x)\right] - \left[\beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} + \beta \log Z(x)\right]$$

Notice that partition function $\beta \log Z(x)$ **cancels out completely**:

$$r(x, y_w) - r(x, y_l) = \beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}$$

6. Final DPO Loss Function : Taking negative log-likelihood over preference dataset $D = \{(x, y_w, y_l)\}$:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim D} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

Why DPO is revolutionary: Eliminates the need to train a separate reward model or run unstable PPO reinforcement learning!

APPENDIX: Standard Formulas & Foundational Identities

1. Chain Rule of Probability: Step-by-Step Derivation

The Chain Rule permits factorizing any joint probability distribution over N random variables into a product of conditional probabilities.

Fundamental Definition (2 Variables, $N = 2$): For any two random variables x_1 and x_2, the definition of conditional probability is:

$$p(x_2 \mid x_1) = \frac{p(x_1, x_2)}{p(x_1)} \implies p(x_1, x_2) = p(x_1)\, p(x_2 \mid x_1)$$

Complete Step-by-Step Derivation for 3 Variables ($N = 3$): We wish to derive the joint probability $p(x_1, x_2, x_3)$.

(a) **Step 1 (First Factorization)**: Treat the pair (x_1, x_2) as a single composite variable $Y = (x_1, x_2)$. Apply the 2-variable definition between Y and x_3:

$$p(x_1, x_2, x_3) = p(Y, x_3) = p(Y)\, p(x_3 \mid Y) = p(x_1, x_2)\, p(x_3 \mid x_1, x_2)$$

(b) **Step 2 (Second Factorization)**: Apply the 2-variable definition to expand the marginal term $p(x_1, x_2)$:

$$p(x_1, x_2) = p(x_1)\, p(x_2 \mid x_1)$$

(c) **Step 3 (Substitution & Final Equation)**: Substitute $p(x_1, x_2) = p(x_1)\, p(x_2 \mid x_1)$ from Step 2 into the Step 1 equation:

$$\begin{aligned} p(x_1, x_2, x_3) &= [p(x_1)\, p(x_2 \mid x_1)] \cdot p(x_3 \mid x_1, x_2) \\ &= \mathbf{p(x_1)\, p(x_2 \mid x_1)\, p(x_3 \mid x_1, x_2)} \end{aligned}$$

General N Variables ($N \geq 2$): By repeating this substitution recursively for N variables:

$$p(x_1, x_2, \ldots, x_N) = p(x_1)\, p(x_2 \mid x_1)\, p(x_3 \mid x_1, x_2) \cdots p(x_N \mid x_1, \ldots, x_{N-1}) = p(x_1) \prod_{i=2}^{N} p(x_i \mid x_1, \ldots, x_{i-1})$$

2. Conditional Probability Tables (CPTs) & Factor Representation

In a Directed Acyclic Graph (DAG) or Belief Network, every variable node x_i is governed by a **Conditional Probability Table (CPT)**, which specifies the probability distribution of x_i given all possible state configurations of its parent nodes $\mathrm{pa}(x_i)$:

$$p(x_i \mid \mathrm{pa}(x_i))$$

- **Root Nodes** ($\mathrm{pa}(x_i) = \emptyset$): The CPT reduces to the prior marginal probability distribution $p(x_i)$.
- **Joint Factorization Rule**: The joint probability distribution over all N variables is computed by multiplying all node CPTs together:

$$p(x_1, x_2, \ldots, x_N) = \prod_{i=1}^{N} p(x_i \mid \mathrm{pa}(x_i))$$

- **Junction Tree Assignment**: Every CPT factor $p(x_i \mid \mathrm{pa}(x_i))$ must be assigned to exactly one clique C_k containing $\{x_i\} \cup \mathrm{pa}(x_i)$, forming initial clique potential $\psi(C_k)$.

3. Bayes' Rule & Law of Total Probability

$$p(y \mid x) = \frac{p(x \mid y)p(y)}{p(x)} = \frac{p(x \mid y)p(y)}{\sum_{y'} p(x \mid y')p(y')}$$

3. Evidence Lower Bound (ELBO) & Jensen's Inequality

$$\log p(x) \geq \mathbb{E}_{q(z|x)}\left[\log \frac{p(x,z)}{q(z \mid x)}\right] = \mathbb{E}_{q(z|x)}[\log p(x \mid z)] - KL(q(z \mid x) \parallel p(z)) \equiv \mathcal{L}_{\text{ELBO}}$$

4. Gaussian Closed-Form KL Divergence

$$KL(q(z \mid x) \parallel p(z)) = -\frac{1}{2}\sum_{j=1}^{d}\left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right)$$

5. Variance Scaling in Self-Attention ($\sqrt{d_k}$)

$$\text{Var}(q \cdot k) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = d_k \implies \text{Var}\left(\frac{q \cdot k}{\sqrt{d_k}}\right) = \frac{d_k}{d_k} = 1$$

6. Direct Preference Optimization (DPO) Loss

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

7. Transformer Matrix Dimensions: Architectural Hyperparameters vs Softmax Scaling

For an input sequence of length N tokens, total model dimension d, and h heads:

- **Distinction Between Hyperparameters and $\sqrt{d_k}$ Derivation**:
  (a) **Architectural Hyperparameter Definition ($d_k = d/h$)**: Engineers set total model dimension d (e.g. 512) and number of heads h (e.g. 8). The head dimension d_k is then determined by $d_k = d/h$ ($512/8 = 64$).
  (b) **Mathematical Softmax Scaling Derivation ($\frac{1}{\sqrt{d_k}}$)**: Once d_k is determined by the architecture, the variance of the dot product $\mathbf{q}\cdot\mathbf{k}$ scales with d_k ($\text{Var}(\mathbf{q}\cdot\mathbf{k}) = d_k$). To prevent large dot products from causing vanishing gradients in Softmax, we divide by $\sqrt{\mathbf{d_k}}$ to normalize variance to $\mathbf{1.0}$.
- **Definition of d (d_{model})**: The total hidden embedding dimension of the Transformer model (each token $\mathbf{x}_i \in \mathbb{R}^d$).
- **Definition of d_k (k)**: The projection dimension of Queries and Keys per attention head ($d_k = d/h$).
- **How d is Chosen**: Selected based on compute budget ($d = 512, 1024, 4096, 8192$).
- **How h is Chosen**: Selected to divide d into standard head dimensions $d_k \in \{64, 128\}$, enabling multi-subspace attention diversity across heads.
- $\mathbf{Q} \in \mathbb{R}^{N \times d_k}$: Query matrix ($\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$). Row $\mathbf{q}_i$ is what token i searches for.
- $\mathbf{K} \in \mathbb{R}^{N \times d_k}$: Key matrix ($\mathbf{K} = \mathbf{X}\mathbf{W}_K$). Row $\mathbf{k}_j$ is what token j offers.
- $\mathbf{V} \in \mathbb{R}^{N \times d_v}$: Value matrix ($\mathbf{V} = \mathbf{X}\mathbf{W}_V$). Row $\mathbf{v}_j$ is token j's information payload.
- $\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \in \mathbb{R}^{N \times N}$: Pairwise token attention weights.
- $\mathbf{O} = \mathbf{A}\mathbf{V} \in \mathbb{R}^{N \times d_v}$: Updated contextual output representation for each token i.

Symbol	Name	Shape (Color-Coded Example)	Interpretation in Terms of Tokens
N	Sequence Length	$\mathbf{N = 512}$	Total number of tokens in the prompt sequence (512 tokens).
d (d_{model})	Embedding Size	$\mathbf{d = 4096}$	Hidden vector dimension per token ($\mathbf{X} \in \mathbb{R}^{\mathbf{512}\times\mathbf{4096}}$ has 512 token rows $\mathbf{x}_i \in \mathbb{R}^{4096}$).
d_k	Head Projection	$\mathbf{d_k = 128}$	Dimension of Query/Key vectors per head ($d_k = d/h = 4096/32 = 128$).
$\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$	Weights	$\mathbf{4096} \times \mathbf{128}$	Projection matrices mapping 4096-dim embeddings to 128-dim head space.
$\mathbf{Q}$	Query Matrix	$\mathbf{512} \times \mathbf{128}$	$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$. Row $\mathbf{q}_i \in \mathbb{R}^{128}$ is **what token i searches for**.
$\mathbf{K}$	Key Matrix	$\mathbf{512} \times \mathbf{128}$	$\mathbf{K} = \mathbf{X}\mathbf{W}_K$. Row $\mathbf{k}_j \in \mathbb{R}^{128}$ is **what token j offers to others**.
$\mathbf{V}$	Value Matrix	$\mathbf{512} \times \mathbf{128}$	$\mathbf{V} = \mathbf{X}\mathbf{W}_V$. Row $\mathbf{v}_j \in \mathbb{R}^{128}$ is **token j's content payload**.
$\mathbf{S}$	Score Matrix	$\mathbf{512} \times \mathbf{512}$	Scaled dot products $S_{ij} = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{\mathbf{128}}}$ between all 512×512 token pairs ($\sqrt{128} \approx 11.31$).
$\mathbf{A}$	Attention Weights	$\mathbf{512} \times \mathbf{512}$	Softmax matrix $\mathbf{A} = \text{softmax}(\mathbf{S})$. Row i sums to 1.0, giving weights token i assigns to all 512 tokens.
$\mathbf{O}_{\text{head}}$	Head Output	$\mathbf{512} \times \mathbf{128}$	Contextual output $\mathbf{O} = \mathbf{A}\mathbf{V}$. Row $\mathbf{o}_i = \sum_{j=1}^{512} A_{ij}\mathbf{v}_j \in \mathbb{R}^{128}$ is updated contextual representation for token i.